

CO5-AT2
MLA0101-FUNDAMENTALS OF MACHINE LEARNING

Annexure A
Pseudocode Writing Task (Algorithm Design)

Question 1: Monte Carlo Sampling for Probability Estimation

A logistics company wants to estimate the probability of on-time deliveries under uncertain traffic conditions using Monte Carlo Sampling.

Task:

Write a pseudocode algorithm to:

- Generate random samples representing delivery conditions.
- Simulate multiple delivery trials.
- Estimate the probability of on-time delivery.
- Display the estimated probability after all simulations.

Question 2: ϵ -Greedy Algorithm for K-Armed Bandit

An online streaming platform recommends one of several movies to maximize user engagement. The platform must balance exploration and exploitation.

Task:

Write a pseudocode algorithm to:

- Initialize rewards for all movie recommendations.
- Implement the ϵ -greedy exploration strategy.
- Update reward estimates after each user interaction.
- Identify the movie with the highest expected reward.

Question 3: Value Iteration for Robot Path Planning

A warehouse robot must determine the optimal path to reach a destination while maximizing rewards and avoiding obstacles.

Task:

Write a pseudocode algorithm to:

- Initialize the value function for all states.
- Update state values using the Bellman optimality equation.
- Repeat the process until convergence.
- Output the optimal value function and policy.

Question 4: Temporal Difference (TD) Learning for Game Score Prediction

A game AI agent learns the value of each game state based on rewards received after every move.

Task:

Write a pseudocode algorithm to:

- Initialize state-value estimates.
- Observe the current state, reward, and next state.
- Update the state value using the Temporal Difference (TD) learning rule.
- Continue until all game episodes are completed.

Question 5: Policy Iteration for Autonomous Drone Navigation

An autonomous drone must learn the optimal navigation policy to deliver packages while minimizing energy consumption.

Task:

Write a pseudocode algorithm to:

- Initialize a random policy.
 - Perform policy evaluation to compute state values.
 - Improve the policy by selecting the best action for each state.
 - Repeat until the policy becomes stable and output the optimal policy.
-

1: MONTE CARLO SAMPLING FOR PROBABILITY ESTIMATION

Problem

Estimate the probability of **on-time delivery** when delivery conditions are uncertain.

Pseudocode

ALGORITHM MonteCarloDeliveryProbability

INPUT:

N = number of simulations

TrafficConditions = possible traffic conditions

Probability[condition] = probability of each traffic condition

DeliveryTime[condition] = delivery time under each condition

Deadline = maximum allowed delivery time

OUTPUT:

Estimated probability of on-time delivery

BEGIN

OnTimeCount $\leftarrow$ 0

FOR trial $\leftarrow$ 1 TO N DO

// Generate a random delivery condition

condition $\leftarrow$ RANDOM_CHOICE(TrafficConditions, Probability)

// Simulate delivery

time $\leftarrow$ DeliveryTime[condition]

// Check whether delivery is on time

IF time $\leq$ Deadline THEN

OnTimeCount $\leftarrow$ OnTimeCount + 1

END IF

END FOR

EstimatedProbability $\leftarrow$ OnTimeCount / N

DISPLAY "Estimated Probability of On-Time Delivery = ",
EstimatedProbability

END

Example

If: N = 10,000

OnTimeCount = 8,700

Then:

$$P(\text{OnTime}) = \frac{8700}{10000} = 0.87$$

Therefore:

$$P(\text{OnTime}) = 87\%$$

Key idea

More simulations generally make the Monte Carlo estimate more stable and closer to the true probability.

2: E-GREEDY ALGORITHM FOR K-ARMED BANDIT

Problem

A streaming platform has several movies and wants to maximize user engagement while balancing exploration and exploitation.

Pseudocode

ALGORITHM EpsilonGreedyBandit

INPUT:

K = number of movies

N = number of user interactions

ϵ = exploration probability

OUTPUT:

Movie with highest estimated expected reward

BEGIN

```

FOR movie  $\leftarrow$  1 TO K DO
    RewardSum[movie]  $\leftarrow$  0
    Count[movie]  $\leftarrow$  0
    EstimatedReward[movie]  $\leftarrow$  0
END FOR
FOR interaction  $\leftarrow$  1 TO N DO
    r  $\leftarrow$  RANDOM_NUMBER(0, 1)
    IF r <  $\epsilon$  THEN
        // Exploration
        movie  $\leftarrow$  RANDOM_CHOICE(1, K)
    ELSE
        // Exploitation
        movie  $\leftarrow$  ARGMAX(EstimatedReward)
    END IF
    // Show selected movie and observe user engagement
    reward  $\leftarrow$  GET_USER_REWARD(movie)
    // Update statistics
    RewardSum[movie]  $\leftarrow$  RewardSum[movie] + reward
    Count[movie]  $\leftarrow$  Count[movie] + 1
    EstimatedReward[movie]  $\leftarrow$ 
        RewardSum[movie] / Count[movie]
END FOR
BestMovie  $\leftarrow$  ARGMAX(EstimatedReward)
DISPLAY "Movie with Highest Expected Reward = ", BestMovie
DISPLAY "Expected Reward = ", EstimatedReward[BestMovie]
END

```

ϵ -Greedy Logic

If:

$$\epsilon = 0.1$$

then approximately:

- **10%** $\rightarrow$ explore random movies
- **90%** $\rightarrow$ exploit the movie with the highest estimated reward

Reward Example

If a user watches/clicks/engages with a movie:

$$Reward = 1$$

Otherwise:

$$Reward = 0$$

For a movie receiving:

10 interactions

7 positive rewards

its estimated reward is:

$$\frac{7}{10} = 0.7$$

3: VALUE ITERATION FOR ROBOT PATH PLANNING

Problem

A warehouse robot must find an optimal path while avoiding obstacles and maximizing rewards.

Pseudocode

ALGORITHM ValueIteration

INPUT:

States = all robot positions
Actions = {UP, DOWN, LEFT, RIGHT}
Reward(s,a) = reward for taking action a in state s
Transition(s,a,s') = probability of reaching s'
 γ = discount factor
 θ = convergence threshold
Goal = destination state
Obstacles = blocked states

OUTPUT:

Optimal value function V
Optimal policy π

BEGIN

// Initialize value function

FOR each state s IN States DO

IF s = Goal THEN

V[s] $\leftarrow$ 0

ELSE IF s IN Obstacles THEN

V[s] $\leftarrow -\infty$

ELSE

V[s] $\leftarrow$ 0

END IF

END FOR

REPEAT

$\Delta \leftarrow 0$

FOR each state s IN States DO

IF s = Goal OR s IN Obstacles THEN

CONTINUE

END IF

OldValue $\leftarrow$ V[s]

BestValue $\leftarrow -\infty$

FOR each action a IN Actions DO

ExpectedValue $\leftarrow$ 0

FOR each possible next state s' DO

ExpectedValue $\leftarrow$ ExpectedValue +

Transition(s,a,s') *

(Reward(s,a) + γ * V[s'])

END FOR

IF ExpectedValue > BestValue THEN

BestValue $\leftarrow$ ExpectedValue

END IF

END FOR

V[s] $\leftarrow$ BestValue

$\Delta \leftarrow \text{MAX}(\Delta, \text{ABS}(\text{OldValue} - \text{V[s]}))$

```

    END FOR
UNTIL  $\Delta < \theta$ 
// Extract optimal policy
FOR each state s IN States DO
    IF s = Goal OR s IN Obstacles THEN
        CONTINUE
    END IF
     $\pi[s] \leftarrow$  action having maximum expected value
END FOR
DISPLAY "Optimal Value Function:"
DISPLAY V
DISPLAY "Optimal Policy:"
DISPLAY  $\pi$ 
END

```

Bellman Optimality Equation

The important equation used is:

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s', | sa) V(s') \right]$$

Termination

The algorithm stops when:

$$\Delta < \theta$$

This means that the value function has approximately converged.

4: TEMPORAL DIFFERENCE (TD) LEARNING FOR GAME SCORE PREDICTION

Problem

A game AI learns the value of each state from rewards received after every move.

Pseudocode

ALGORITHM TD_Learning

INPUT:

Episodes = number of game episodes

α = learning rate

γ = discount factor

OUTPUT:

Learned state-value function V

BEGIN

// Initialize state values

FOR each state s DO

$V[s] \leftarrow 0$

END FOR

FOR episode $\leftarrow 1$ TO Episodes DO

Initialize game

state $\leftarrow$ INITIAL_STATE()

WHILE state is not terminal DO

// Agent takes an action

action $\leftarrow$ SELECT_ACTION(state)

// Observe reward and next state

reward, nextState $\leftarrow$ TAKE_ACTION(action)

// TD target

```

    TD_Target ← reward +  $\gamma$  * V[nextState]
    // TD error
    TD_Error ← TD_Target - V[state]
    // Update state value
    V[state] ← V[state] +  $\alpha$  * TD_Error
    // Move to next state
    state ← nextState
  END WHILE
END FOR
DISPLAY "Learned State Values:"
DISPLAY V
END

```

TD Learning Formula

The TD update rule is:

$$V(s) \leftarrow V(s) + \alpha[R + \gamma V(s') - V(s)]$$

Where:

- $V(s)$ = current state value
- R = immediate reward
- $V(s')$ = next-state value
- α = learning rate
- γ = discount factor

Example

Suppose:

$$\begin{aligned}
 V(s) &= 5 \\
 R &= 2 \\
 V(s') &= 8 \\
 \alpha &= 0.1 \\
 \gamma &= 0.9
 \end{aligned}$$

Then:

$$\begin{aligned}
 V(s) &= 5 + 0.1[2 + (0.9)(8) - 5] \\
 &= 5 + 0.1[2 + 7.2 - 5] \\
 &= 5 + 0.42 \\
 \boxed{V(s) = 5.42}
 \end{aligned}$$

5: POLICY ITERATION FOR AUTONOMOUS DRONE NAVIGATION

Problem

An autonomous drone must find the optimal navigation policy while minimizing energy consumption.

Pseudocode

ALGORITHM PolicyIteration

INPUT:

States = all drone positions
 Actions = possible navigation actions
 Reward(s,a) = reward/cost of action
 Transition(s,a,s') = transition probabilities
 γ = discount factor
 θ = convergence threshold

OUTPUT:

Optimal policy

```

BEGIN
  // Step 1: Initialize random policy
  FOR each state s IN States DO
     $\pi[s] \leftarrow \text{RANDOM\_CHOICE}(\text{Actions})$ 
  END FOR
  REPEAT
    // -----
    // Policy Evaluation
    // -----
    FOR each state s IN States DO
       $V[s] \leftarrow 0$ 
    END FOR
    REPEAT
       $\Delta \leftarrow 0$ 
      FOR each state s IN States DO
        OldValue  $\leftarrow V[s]$ 
         $a \leftarrow \pi[s]$ 
         $V[s] \leftarrow$ 
          SUM over all s' [
            Transition(s,a,s') *
            (Reward(s,a) +  $\gamma$  *  $V[s']$ )
          ]
         $\Delta \leftarrow \text{MAX}(\Delta, \text{ABS}(\text{OldValue} - V[s]))$ 
      END FOR
    UNTIL  $\Delta < \theta$ 
    // -----
    // Policy Improvement
    // -----
    PolicyStable  $\leftarrow \text{TRUE}$ 
    FOR each state s IN States DO
      OldAction  $\leftarrow \pi[s]$ 
      BestAction  $\leftarrow \text{NONE}$ 
      BestValue  $\leftarrow -\infty$ 
      FOR each action a IN Actions DO
        ActionValue  $\leftarrow$ 
          SUM over all s' [
            Transition(s,a,s') *
            (Reward(s,a) +  $\gamma$  *  $V[s']$ )
          ]
        IF ActionValue > BestValue THEN
          BestValue  $\leftarrow$  ActionValue
          BestAction  $\leftarrow$  a
        END IF
      END FOR
       $\pi[s] \leftarrow \text{BestAction}$ 
      IF OldAction  $\neq \pi[s]$  THEN
        PolicyStable  $\leftarrow \text{FALSE}$ 
      END IF
    END FOR
  UNTIL PolicyStable = TRUE

```


DISPLAY "Optimal Policy:"
 DISPLAY π
 END

Important Difference: Value Iteration vs Policy Iteration

This is useful if your examiner asks you orally.

Value Iteration	Policy Iteration
Directly improves value function	Alternates between policy evaluation and improvement
Uses Bellman optimality equation	First evaluates current policy
Extracts policy after convergence	Improves policy after evaluation
Usually simpler pseudocode	More structured two-step process
Q3	Q5

Value Iteration

$$V(s) \leftarrow \max_a \left[R + \gamma \sum P V(s') \right]$$

Policy Iteration

Step 1: Policy Evaluation

$$V^\pi(s) = R + \gamma \sum P(s', | s \pi(s)) V^\pi(s')$$

Step 2: Policy Improvement

$$\pi^{\text{new}}(s) = \text{argmax} [R + \gamma \sum P(s'|s, a) V(s')]$$
